

LuxLLM-Agent: An Explainable LLM-Assisted Agent and Replay Viewer for Lux AI Season 3

First Author

Affiliation / Address line 1
email@domain

Second Author

Affiliation / Address line 1
email@domain

Abstract

We present LuxLLM-Agent, an explainable LLM-assisted agent system for Lux AI Season 3. The system combines a local game-playing agent, rule-based fallback policies, structured decision logs, and an S1-style isometric battle replay viewer for inspecting agent behaviour. Unlike a standard replay viewer, our interface connects game states with decision traces, risk-filter events, match-level scores, and final result summaries. We evaluate the system through controlled matches against a rule-based baseline and a lightweight scalability simulation with up to 1000 worker agents. The demo package includes a replay viewer, screenshots, replay JSON, evaluation reports, and a screencast video. The source package is available at <https://github.com/zewang-liverpool/luxllm-agent-demo>, and the screencast video is available at <https://github.com/zewang-liverpool/luxllm-agent-demo/releases/tag/demo-video>.

1 Introduction

Game AI competitions provide a useful testbed for studying decision making, planning, and multi-agent behaviour under partial observability and limited runtime. Lux AI Season 3 is especially challenging because an agent must explore an initially uncertain map, reason about relics and energy, control multiple units, and react to an opponent over many steps. Large language models (LLMs) can provide flexible high-level reasoning, but directly using them inside a real-time game loop introduces latency, instability, and reproducibility concerns.

This paper presents LuxLLM-Agent, a system demonstration that focuses on making LLM-assisted game agents inspectable and reproducible. The system uses an LLM-assisted strategy layer with rule-based fallback, records structured decision traces, and provides a replay viewer that

visualises both the battle state and the agent’s reasoning evidence. The target users are researchers and students interested in LLM agents, game AI, agent evaluation, and explainable decision systems.

The central motivation is that the outcome of a game match is not enough to understand whether an LLM-assisted agent is behaving reliably. A final score does not show whether the LLM decision was fresh or stale, whether a rule fallback was used, whether a target was changed by a safety filter, or whether the agent followed a coherent strategy over time. For this reason, LuxLLM-Agent treats replay visualisation as part of the agent evaluation pipeline rather than as a separate cosmetic interface.

Our contributions are:

- an LLM-assisted Lux AI Season 3 agent with cached strategy decisions and rule-based fallback for runtime stability;
- a log-driven explanation pipeline that records agent strategies, fallback events, risk-filter changes, and match-level outcomes;
- an S1-style isometric battle replay viewer that turns Lux S3 logs into an inspectable replay interface for demonstration and paper figures;
- a controlled evaluation and lightweight scalability simulation showing how sparse LLM-generated policy templates can be amortised across many worker agents.

2 System Overview

LuxLLM-Agent consists of four main components: an LLM-assisted agent, a rule-based fallback policy, a structured logging and evaluation pipeline, and a replay-driven visualisation interface. During a match, the agent observes the current game state and converts it into a compact decision context. When the LLM is available within the runtime budget, it produces a high-level strategy such as

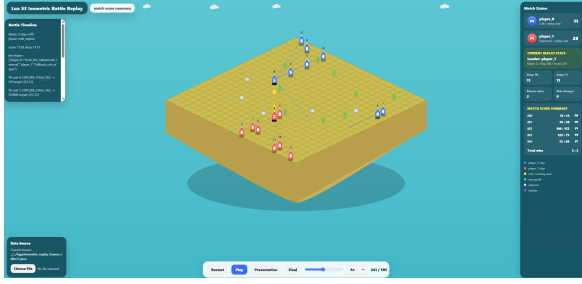


Figure 1: S1-style isometric battle replay viewer for Lux AI Season 3. The viewer loads merged replay frames and shows both players, unit positions, score, match state, timeline, and per-match score summary.

exploration, energy collection, relic movement, or scoring. If the LLM call is unavailable, stale, or unsafe, the system falls back to deterministic rule policies.

The system is designed around traceability. Each important decision is logged with the current strategy, decision source, fallback status, unit action, and risk-filter information. These logs are later converted into replay frames and evaluation summaries. The replay viewer then reconstructs a visual battle timeline from these frames, allowing users to inspect how the agent behaved over time rather than only seeing the final score.

The end-to-end data flow is intentionally simple. A local Lux AI Season 3 match produces console logs and structured frame logs. A replay-frame builder merges the relevant per-step evidence into a compact JSON representation. The viewer loads this JSON file and renders the game state as a floating-island battle map. Finally, screenshots, verification reports, and artifact indices are generated from the same replay package so that the paper, demo video, and repository all refer to the same evidence source.

2.1 Implementation

The prototype is implemented in Python and JavaScript. The Lux AI Season 3 agent runs locally through the competition environment, while LLM decisions are produced through a local Ollama backend using Qwen2.5 models. The implementation records console logs, structured JSONL frame logs, evaluation JSON files, replay frames, screenshots, and Markdown summaries. The viewer is implemented as a standalone HTML interface that can be opened through a local HTTP server and used for live demonstration or video recording.

The current demonstration package includes a

stable S3 isometric battle replay viewer, merged replay JSON, four paper-ready screenshots, a demo video, controlled evaluation summaries, and scalability reports. The system does not require cloud LLM APIs for the local demonstration. This makes the artifact suitable for reviewer inspection because the replay viewer can be launched locally without rerunning the full game environment or making external model calls.

2.2 Design Choices

The system separates high-level strategy generation from low-level action execution. This choice is important because a full Lux AI Season 3 match requires many small unit-level decisions, while the LLM is better suited to sparse strategic reasoning. The rule-based fallback layer ensures that the agent can continue acting even when the LLM is unavailable, slow, or produces an unusable response. This makes the system more robust than a design that depends on an LLM for every action.

The viewer is designed in the style of the Season 1 floating-island replay interface so that the Season 3 demonstration remains visually consistent with earlier Lux-style visualisations. At the same time, the interface adds Season 3-specific evidence such as match score summaries, final result overlays, known relic counts, risk-change counts, and replay-frame coverage. The result is a viewer that is both presentation-friendly and useful for debugging agent behaviour.

3 Demonstration Walkthrough

A typical demonstration begins by opening the S3 isometric battle replay viewer. The user can scrub through the match timeline, observe both player_0 and player_1 units, inspect the current score, and view the per-match score summary. The left panel presents a compact battle timeline, while the right panel shows match status, ship counts, known relics, risk changes, and match-level scores.

The demonstration is organised around a short reviewer-facing workflow. First, the viewer loads the replay JSON and renders the initial floating-island map. Second, the user plays or scrubs through the timeline to observe how both players move across the map. Third, the user switches to Presentation Mode, which reduces auxiliary panels and makes the map easier to record in a short screencast. Fourth, the user inspects the match score summary to see how individual matches con-

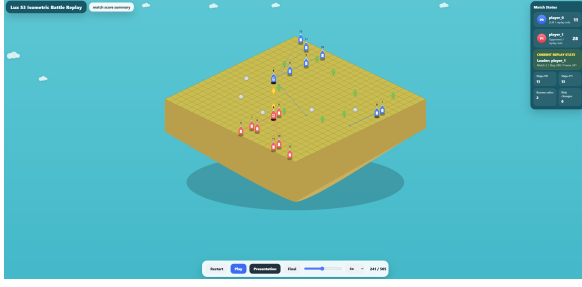


Figure 2: Recording-friendly Presentation Mode of the S3 battle replay viewer. Auxiliary panels are reduced so that the central floating-island battle map becomes the visual focus for demo-video recording.

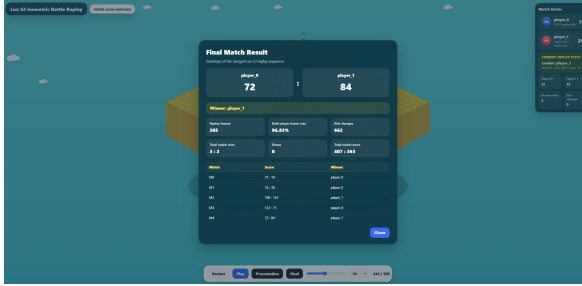


Figure 3: Final result overlay in the S3 replay viewer. The overlay summarises the final winner, final score, replay-frame coverage, total risk-filter changes, total match wins, and total score.

tribute to the overall result. Finally, the user opens the final result overlay to view the winner, score, replay coverage, and risk-filter statistics.

This workflow is designed for both research inspection and demonstration. A researcher can use the viewer to locate suspicious behaviour, fallback-heavy intervals, or sudden score changes. A supervisor or reviewer can use the same viewer to understand the system without reading raw logs. The paper screenshots and screencast are generated from this interface, so the visual evidence in the paper is directly connected to the downloadable artifact.

4 Evaluation

We evaluate the system from three perspectives: competitive match outcome, replay evidence quality, and architecture-level scalability. First, controlled matches compare LLM-assisted agents against a rule-based baseline. Second, replay-frame verification confirms that the viewer can load merged S3 replay data and visualise both players over the match timeline. Third, a lightweight scalability simulation tests whether shared policy templates can be executed by many worker agents

Configuration	Matches	Wins	Win Rate
Rule vs Rule	19	9	47.37%
Qwen2.5 1.5B vs Rule	19	14	73.68%
Qwen2.5 7B vs Rule	19	13	68.42%

Table 1: Controlled win-rate comparison across rule and LLM-assisted configurations.

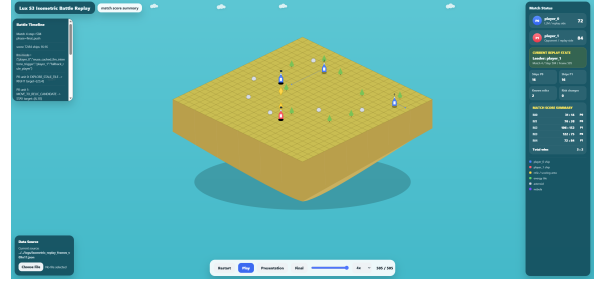


Figure 4: Per-match score summary in the S3 replay viewer. The panel displays individual match outcomes and total match wins over the replay sequence.

without invoking the LLM once per worker.

In the controlled match setting, 57 matches were run across three configurations: rule-based baseline versus rule-based baseline, Qwen2.5 1.5B versus rule baseline, and Qwen2.5 7B versus rule baseline. The smaller Qwen2.5 1.5B configuration achieved the best balance between win rate and runtime stability, with a win rate of 73.68

The verified S3 replay package contains 505 replay frames, with both players present in 96.83

The match results suggest that a smaller local model can be more practical than a larger model under runtime constraints. The 1.5B model is less expressive than the 7B model, but it is also cheaper to call and easier to integrate into a time-sensitive game loop. In this setting, the best-performing configuration is therefore not simply the largest model. Instead, the best configuration is the one that balances strategic usefulness, fallback frequency, and runtime stability.

4.1 Scalability Simulation

To evaluate scalability without overstating full-game throughput, we implemented a synthetic lightweight-worker simulation. The runner instantiates 10, 100, 500, and 1000 worker agents, each executing cached policy templates for 100 decision steps. The simulation does not launch full Lux AI Season 3 matches and does not invoke an LLM for every worker. Instead, five synthetic LLM calls represent sparse strategy-template generation. The 1000-worker setting completes 100,000 light-

Agents	Decisions	LLM Calls	Runtime (s)	Memory (MB)
10	1,000	5	0.008	0.346
100	10,000	5	0.079	1.214
500	50,000	5	0.409	5.072
1000	100,000	5	0.807	9.894

Table 2: Architecture-level scalability simulation using lightweight worker agents. The runner uses cached policy templates and does not invoke an LLM once per worker.

weight decisions in 0.807 seconds, while the LLM-call count remains fixed at 5. This result supports the intended scaling mechanism: expensive strategic reasoning is amortised across many low-cost worker decisions.

This scalability result should be interpreted carefully. It is an architecture-level lightweight-worker simulation rather than a full Lux benchmark. The reported runtime measures lightweight policy execution and synthetic worker-state updates, not game-engine stepping, rendering, or 1000 complete Lux matches. The purpose of this experiment is to test the system design claim that sparse LLM-generated policy templates can be reused across many worker agents, rather than to claim that the full Lux environment has been run with 1000 complete LLM-controlled players.

5 Comparison with Existing Systems

Official Lux replay tools are useful for observing game outcomes, but they are not designed to explain why an LLM-assisted agent selected a strategy, when fallback was triggered, or how risk filtering changed unit targets. Standard console logs provide decision evidence, but they are difficult to interpret visually and do not provide a replay-like user experience.

LuxLLM-Agent fills this gap by combining agent decision traces, match evaluation, and replay visualisation. The system is not only a game-playing agent and not only a viewer. It is a demonstration pipeline that connects LLM strategy decisions, rule fallback, risk-filter events, match outcomes, and visual replay evidence. This makes it suitable for analysing LLM-assisted game agents in a way that is more inspectable than raw logs and more explanatory than a standard replay.

Compared with a pure game-playing submission, the system emphasises reproducible inspection. Compared with a pure visual replay tool, it preserves decision evidence from the agent pipeline. Compared with a per-agent LLM controller, it uses

cached strategy templates and rule execution so that the LLM plays a sparse strategic role instead of controlling every unit action. This positioning is important for demo-track evaluation because the contribution is not only a game result, but also an inspectable system for studying LLM-assisted game agents.

6 Licensing and Availability

The project is intended to be released as a downloadable research prototype for local demonstration and reproducibility. The artifact package includes the S3 isometric battle replay viewer, merged replay JSON, screenshots, a screencast video, controlled-evaluation summaries, and scalability reports. The main demonstration artifacts include:

- the S3 isometric battle replay viewer;
- the merged replay-frame JSON used by the viewer;
- four paper-ready screenshots of the replay viewer, presentation mode, final result overlay, and match score summary;
- a screencast video of the demonstration;
- controlled match-evaluation reports and lightweight scalability-simulation reports.

The source code and downloadable artifact package are available at <https://github.com/zewang-liverpool/luxllm-agent-demo>. The screencast video is available through the project release page at <https://github.com/zewang-liverpool/luxllm-agent-demo/releases/tag/demo-video>.

7 Conclusion

LuxLLM-Agent demonstrates an explainable LLM-assisted agent pipeline for Lux AI Season 3. The system combines local LLM strategy generation, rule-based fallback, structured decision logging, replay-driven visualisation, controlled evaluation, and lightweight scalability evidence. The results suggest that smaller local LLMs can be effective under runtime constraints, and that sparse LLM-generated policy templates can be reused across many lightweight worker agents. The demonstration is intended to help researchers and students inspect, reproduce, and discuss LLM-assisted game-agent behaviour.

Ethics Statement

This system is developed for research and educational use in a game AI environment. It does not use personal data and does not make decisions about real users. The main risks are overclaiming system capability and misinterpreting synthetic scalability results as full-game multi-agent performance. We mitigate these risks by clearly separating controlled match evaluation, replay-viewer evidence, and architecture-level scalability simulation. The system should be presented as a research prototype rather than a deployed autonomous decision system.

Acknowledgements

The author thanks the project supervisor and the Lux AI Challenge community for providing the competition environment and motivation for this work.

A Additional Details

The local development artifacts include replay JSON, Markdown reports, screenshots, demo video, and LaTeX integration files. The final S3 replay viewer was verified with 505 replay frames, 96.83